

LeetCode編碼：

2335

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔



題目 (含解釋&說明)



給定一個長度為3的陣列(**amount**)，分別代表冷水、溫水和熱水各自需要裝滿的杯數。每秒可以選擇同時裝兩杯不同溫度的水，或者只裝一杯水，求出裝滿所有杯子所需的最少秒數。

- 例如 **amount** = [1, 4, 2] 時：第 1 秒同時裝溫水與熱水，陣列變為 [1, 3, 1]；第 2 秒同時裝冷水與溫水，變為 [0, 2, 1]；第 3 秒再裝溫水與熱水，變為 [0, 1, 0]；第 4 秒因為只剩溫水，單獨裝完就結束，因此共需 4 秒。
- 邊界有幾個情況，當只剩下一種溫度的水時，因為無法配對所以剩下幾杯就需要幾秒。第二，如果最大杯數大於或等於其餘兩者之和，因為其他水不管怎麼配對都無法完全消耗掉最大的那一項，所以最少秒數就會直接等於最大杯數。如果三種水的總杯數加起來是奇數，代表最後必定會剩下 1 杯水需單獨裝，因此須取 **Ceiling**。



- 解法一：**Greedy + Max-Heap**

- 貪心策略（**Greedy**）：讓每秒的裝填進度最大化，最直覺且有效的策略就是每次都優先選擇當前需求量最多的兩種飲料來消耗，如果不及早處理數量最多的飲料，最後可能會無法被配對。

每輪優先消耗最大代價的邏輯，與 **LeetCode 621** 以及 **Huffman Coding** 的核心思想相似。

- 再來使用**Max-Heap**，因為每一輪都需要快速取出前兩個最大值，而 **Heap** 可以在 $O(\lg n)$ 的時間內完成這個操作。由於 **Python** 內建的 **heapq** 模組預設是 **Min-Heap**，因此將所有杯數取負號後再存入模擬最大堆疊的行為。
- 每輪迴圈中，我們會從堆疊中取出當前最大的兩個值並各減 **1**，接著再放回堆疊中，直到第二大的值變成 **0** 為止。這代表此時只剩下一種飲料還有需求，已經無法再進行配對，因此我們只需將剩餘的杯數直接加進總秒數即可。

這方法的優點在於邏輯直覺，且能推廣到 **k** 種飲料的場景不受種類數量限制。本題因為堆疊的大小固定為 **3**， $\log 3$ 可視為常數。

但這個解法的缺點是整體時間複雜度為 $O(n)$ （其中 **n** 為總杯數），因為迴圈的執行次數會等於最終答案值。如果飲料杯數非常龐大這個方法的執行效率就會明顯下降。



- 解法二：數學公式解 $O(1)$
- 觀察「杯數分布」的極端情況推導出直接計算的數學公式，與 LeetCode 1953 相同，屬於消耗類貪心問題的通用結論。
- 將三種飲料由小到大排序記為 $a \leq b \leq c$ ，並分為兩種情況討論。
情況一（最大值主導）：當 $a + b \leq c$ 時，代表另外兩種飲料加總都不夠與最大項 c 配對。每一秒都讓 c 與 a 或 b 配對，直到 a 和 b 消耗完， c 仍會剩下。剩下的只能單獨裝，因此總秒數直接等於最大值 c 。
- 情況二（三者均衡）：三種飲料需求量接近可互相配對。若總杯數（ $a + b + c$ ）為偶數剛好配對完；若為奇數則剩 1 杯需單獨裝。因此答案為總杯數取Ceiling
這方法的優點是時間複雜度僅 $O(1)$ ，不受杯數大小影響。缺點則是公式依賴「恰好 3 種飲料」的特性無法推廣到 k 種。



- 無論哪種解法，核心都是相同的：瓶頸永遠在於數量最多的那一項（最大值）。
若最大值 $\geq$ 其餘兩者之和，總秒數完全由最大值直接決定；否則 每一秒都能盡量兩兩配對，
總秒數改由「總杯數」決定（總和取Ceiling）。

方法	Time	Space
Heap	$O(n \times \log 3)$	$O(1)$
數學公式	$O(1)$	$O(1)$



```

1  class Solution(object):
2      import heapq
3
4      # 解法一：Greedy + Max-Heap
5      import heapq
6      def fillCups_greedy(self, amount):
7          # 取負值是因為 Python 的 heapq 只有 Min-Heap
8          # 負數讓最大值變成堆頂，模擬 Max-Heap 的行為
9          heap = [-x for x in amount]
10         heapq.heapify(heap)
11         seconds = 0
12         while True:
13             a = heapq.heappop(heap)
14             b = heapq.heappop(heap)
15             # b 是第二大值，若它為 0 代表最多只剩一種飲料有需求
16             # 此時無法再並行，跳出迴圈單獨處理剩餘
17             if -b == 0:
18                 heapq.heappush(heap, a)
19                 break
20             # 各加 1 是因為存的是負數，-x+1 等同於讓原本的 x 減 1
21             a += 1; b += 1
22             heapq.heappush(heap, a)
23             heapq.heappush(heap, b)
24             seconds += 1
25         # 堆頂剩餘值就是還沒裝完的那種飲料的杯數
26         # 全為 0 時加 0 不影響結果，不需要額外判斷
27         return seconds + (-heapq.heappop(heap))
28

```

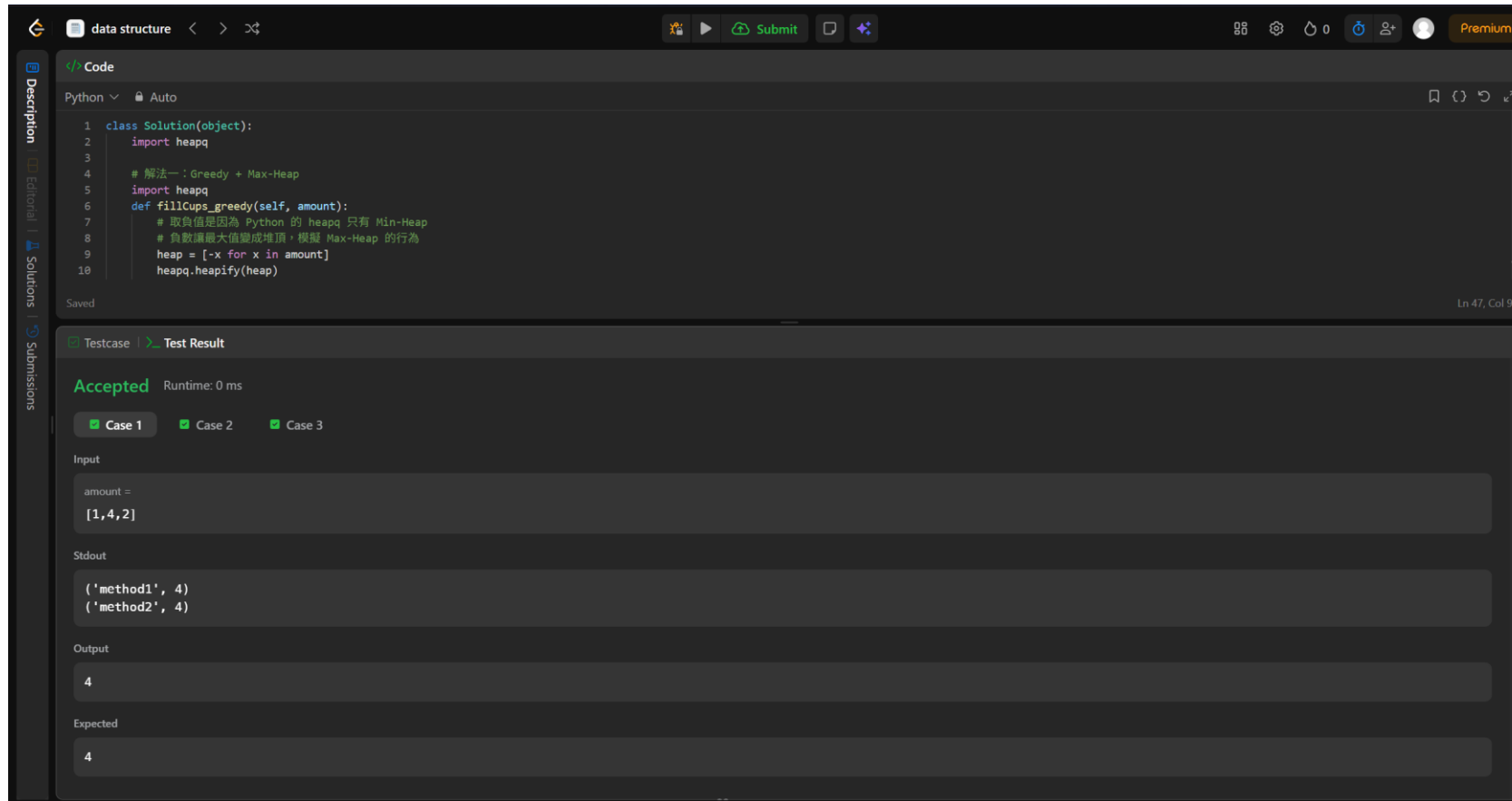
```

29     # 解法二：數學公式  $O(1)$ 
30     def fillCups_math(self, amount):
31         a, b, c = sorted(amount)
32         # a+b == c 時答案仍為 c，因為最大值每秒都需要配對
33         # 其他兩種合計需求恰好能撐滿每一秒，由最大值主導
34         if a + b <= c:
35             return c
36         # 總量為奇數時必有一秒只能裝一杯，+1 確保向上取整
37         # 用整數除法而非 /2 是為了避免浮點誤差
38         return (a + b + c + 1) // 2
39
40     def fillCups(self, amount):
41         res1 = self.fillCups_greedy(amount)
42         res2 = self.fillCups_math(amount)
43
44         print("method1", res1)
45         print("method2", res2)
46
47         return res1

```



- 結果證明：



The screenshot shows a code editor interface with a dark theme. The top bar includes a search icon, a file explorer icon, and a 'Submit' button. The left sidebar contains a 'Description' tab and a 'Solutions' tab. The main editor area displays a Python class named 'Solution' with a method 'fillCups_greedy'. The code includes comments in Chinese explaining the greedy approach and the use of a min-heap. The bottom panel shows the 'Test Result' section, which indicates that the solution is 'Accepted' with a runtime of 0 ms. Three test cases are listed, all of which are passed. The input for Case 1 is 'amount = [1,4,2]', and the output is '4'.

```

1 class Solution(object):
2     import heapq
3
4     # 解法一：Greedy + Max-Heap
5     import heapq
6     def fillCups_greedy(self, amount):
7         # 取負值是因為 Python 的 heapq 只有 Min-Heap
8         # 負數讓最大值變成堆頂，模擬 Max-Heap 的行為
9         heap = [-x for x in amount]
10        heapq.heapify(heap)

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

amount =
[1,4,2]

Stdout

('method1', 4)
('method2', 4)

Output

4

Expected

4



LeetCode編碼：

2073

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔





- n 個人排隊，每個人 i 需要購買 `tickets[i]` 張票。每輪每人只能買 1 張票，買完會自動離隊，沒買完的人則繼續留在隊伍末等待下一輪。給定索引 k ，求第 k 個人買完所有票時，總共幾輪。
- 以 `tickets = [2, 3, 2]`， $k = 2$ 為例，第 k 個人需要買 2 張票。第一輪三人依序各買 1 張後，狀態變為 `[1, 2, 1]`；第二輪 0 號買 1 張後變為 `[0, 2, 1]`，1 號買 1 張變為 `[0, 1, 1]`，接著輪到 2 號買下第 2 張票後離隊，總共花費 6 秒。

注意排在 k 後面的人能參與的輪數不同。

- 排在 k 之前（含 k ）的人，在 k 買完之前最多有機會參與 `tickets[k]` 輪，時間為 `min(tickets[i], tickets[k])`。
- 而排在 k 之後由於 k 買完最後一張票時還沒輪到，實際最多只能參與 `tickets[k] - 1` 輪時間為 `min(tickets[i], tickets[k] - 1)`。
- 當 `tickets[k] == 1` 時， k 在第一輪就離隊，所有排在後面的人時間均為 0。



- 解法一：Queue
- 直接依照題意模擬隊伍排隊，因此使用Queue，運用 `collections.deque` 實作FIFO Queue，因為排隊行為就是一個佇列，每輪從隊頭取出一人處理，若票還沒買完就重新排到隊尾。
- 每輪從佇列取出 (index, 剩餘張數)，將剩餘張數減 1，總時間加 1。
若取出 k 且其剩餘張數剛好歸零，代表 k 買完票，可以直接回傳當前總秒數。
否則若剩餘張數還大於 0，則該人重回佇列尾等待下一輪。
優點是每一步都直接對應題目描述，不需要額外的數學推導。缺點是時間複雜度高達 $O(n \times \text{max_tickets})$ ，當票數上限很大時執行極慢。



- 解法二：數學公式解 $O(n)$
- 從每個人最終實際能買到幾張票的角度尋找規律省去模擬的過程。
- 觀察購票何時結束，由 k 的購票進度決定。
- 排在 k 之前(含 k)，因為在每一輪中都排在 k 前面能持續買，最多能完整參與 $tickets[k]$ 輪，所以時間是 $\min(tickets[i], tickets[k])$ 。
- 排在 k 之後，當 k 在第 $tickets[k]$ 輪買完最後一張票整個流程就停止了，輪不到他們最多只能完成 $tickets[k] - 1$ 輪,時間是 $\min(tickets[i], tickets[k] - 1)$ 。最後將所有人花費的時間加總就是答案。

這個方法的優點是時間複雜度僅需 $O(n)$ ，與票數無關，缺點是需要先想清楚 $i \leq k$ 與 $i > k$ 的邊界,這個「相差 1」不如Queue方式直覺，實作時容易寫錯邊界條件。



方法	Time	Space
Queue 模擬	$O(n \times \text{max_t})$	$O(n)$
數學直算	$O(n)$	$O(1)$



```

1  class Solution(object):
2
3      # 解法一：Queue 模擬
4      from collections import deque
5      def timeRequiredToBuy_queue(self, tickets, k):
6          # 存 (索引, 剩餘張數), 索引是為了判斷何時遇到目標 k
7          queue = deque((i, t) for i, t in enumerate(tickets))
8          time = 0
9          while queue:
10             idx, remain = queue.popleft()
11             time += 1
12             remain -= 1
13             # 只有在 idx == k 且剩餘恰好歸零時才停止
14             # 若只判斷 remain == 0 會在其他人買完時提早結束
15             if idx == k and remain == 0:
16                 return time
17             if remain > 0:
18                 queue.append((idx, remain))
19         return time
20

```

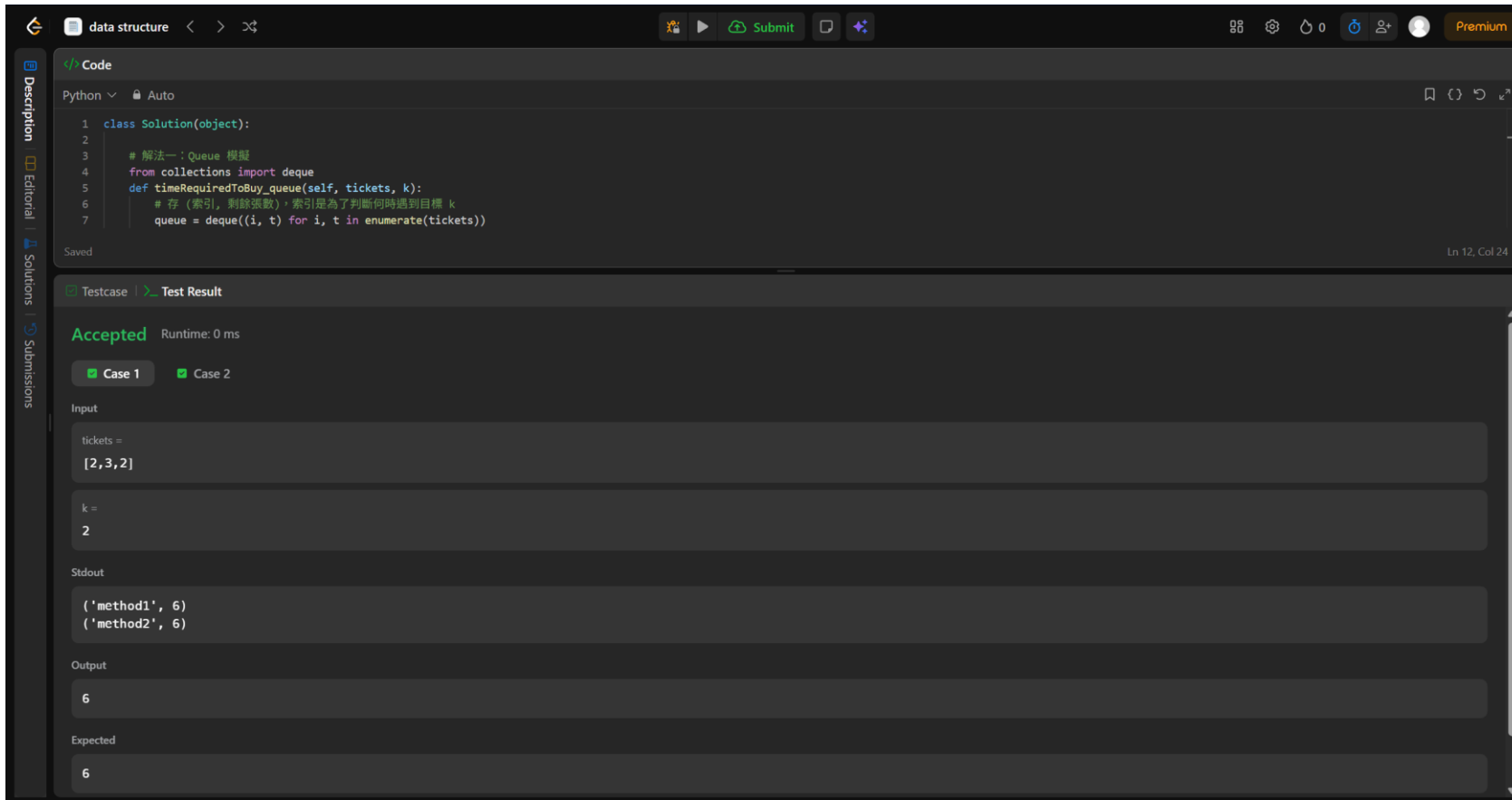
```

21  # 解法二：數學 O(n)
22  def timeRequiredToBuy_math(self, tickets, k):
23      total = 0
24      for i, t in enumerate(tickets):
25          if i <= k:
26              # k 之前 (含 k) 的人每輪都排在 k 前面
27              # 最多能完整買 tickets[k] 輪, 超過的部分 k 已結束所以不算
28              total += min(t, tickets[k])
29          else:
30              # k 之後的人每輪都排在 k 後面
31              # k 買完最後一張時這些人還沒輪到, 少一次機會
32              total += min(t, tickets[k] - 1)
33      return total
34
35  def timeRequiredToBuy(self, tickets, k):
36      res1 = self.timeRequiredToBuy_queue(tickets, k)
37      res2 = self.timeRequiredToBuy_math(tickets, k)
38
39      print("method1", res1)
40      print("method2", res2)
41
42      return res1

```



- 結果證明：



```

1 class Solution(object):
2
3     # 解法一：Queue 模擬
4     from collections import deque
5     def timeRequiredToBuy_queue(self, tickets, k):
6         # 存 (索引, 剩餘張數) * 索引是為了判斷何時遇到目標 k
7         queue = deque((i, t) for i, t in enumerate(tickets))

```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

tickets =
[2, 3, 2]

k =
2

Stdout

('method1', 6)
('method2', 6)

Output

6

Expected

6



LeetCode編碼：

1823

系所：北聯(北醫) 生物醫學工程學系
學號：B812111001
姓名：張博翔





- n 人圍成一圈，依序為 1 到 n ，從 1 號開始順時針數每數到第 k 個人就將其淘汰，下一輪從被淘汰的下一位再開始數。重複此過程直到只剩 1 人，求最後存活者編號。

這是數學史上著名的約瑟夫問題 (Josephus Problem)

- 以 $n = 6, k = 5$ 為例。

第一輪從 1 號開始數 5 個，數到 5 號將其淘汰，圈中剩 [1, 2, 3, 4, 6]。

第二輪從 6 號數 5 個，依序數過 6、1、2、3，第 5 個落在 3 號，剩 [1, 2, 4, 6]。

第三輪從 4 號數 5 個，第 5 個落在 4 號自己繞回來後的下一人，實際上落在 2 號，剩 [1, 4, 6]，第四輪從 4 號數 5 個，落在 6 號剩 [1, 4]。

第五輪從 1 號數 5 個，繞了兩圈後落在 4 號，最後剩下 1 號。

- 考量邊界，當 $k = 1$ 時每輪數到的第 1 人就是起點，等同依序淘汰，最後必為 n 號。當 $k > n$ 時，數超過一圈後取模不影響結果，適用公式。

約瑟夫公式的推導與計算全程在 0-indexed 下進行，是因為餘數運算 ($\% i$) 從 0 開始最自然，最後加 1 才能轉回題目要求的 1-indexed 答案。



- 解法一：Queue
- 利用 `collections.deque` 的 `rotate` 操作來還原圍圈數數過程。
- 每輪的目標是讓第 k 個人移到隊首再淘汰，直觀做法是將前 $k-1$ 個人旋轉到隊尾，第 k 個人移到最前面，再用 `popleft()` 移除，使用 `rotate(-(k-1))` 可以避免管理索引位移時容易出現的 `off-by-one` 錯誤。

優點是直觀還原題目，`rotate` 讓第 k 個人站到最前面這件事一目瞭然。

缺點是時間複雜度為 $O(n \times k)$ ，因為每輪的 `rotate` 本身需要移動 $k-1$ 個元素，當 k 很大時效率極差。



- 解法二：Josephus Problem $O(n)$
- 邏輯是從只有 1 人的情形出發，此時存活者的索引必為 0。
每新增一人，圈子就多了一個人，原本的索引在新的 i 人的圈中需要旋轉 k 格，因此遞推關係為 $f(i) = (f(i-1) + k) \% i$ 。
從 $i = 2$ 一路遞推到 n ，就能得到 n 人圈中存活者的 0-indexed 位置，最後加 1 轉回 1-indexed。
方法優點是 $O(n)$ 時間、 $O(1)$ 空間，且與 k 的大小無關；缺點是公式本身需要數學推導才能理解，直覺不如 Queue 明顯，推導過程需要一定時間。
- 每次淘汰後，存活者的相對位置可以用遞迴來追蹤，不需要真正模擬整個圈，只需要追蹤「若圈中只剩 i 人時，存活者是第幾個」，再逐步還原到 n 人的情境。
縮小問題規模再推回的思維，是動態規劃與數學遞迴問題的核心精神。



方法	Time	Space
Queue	$O(n \times k)$	$O(n)$
約瑟夫公式	$O(n)$	$O(1)$



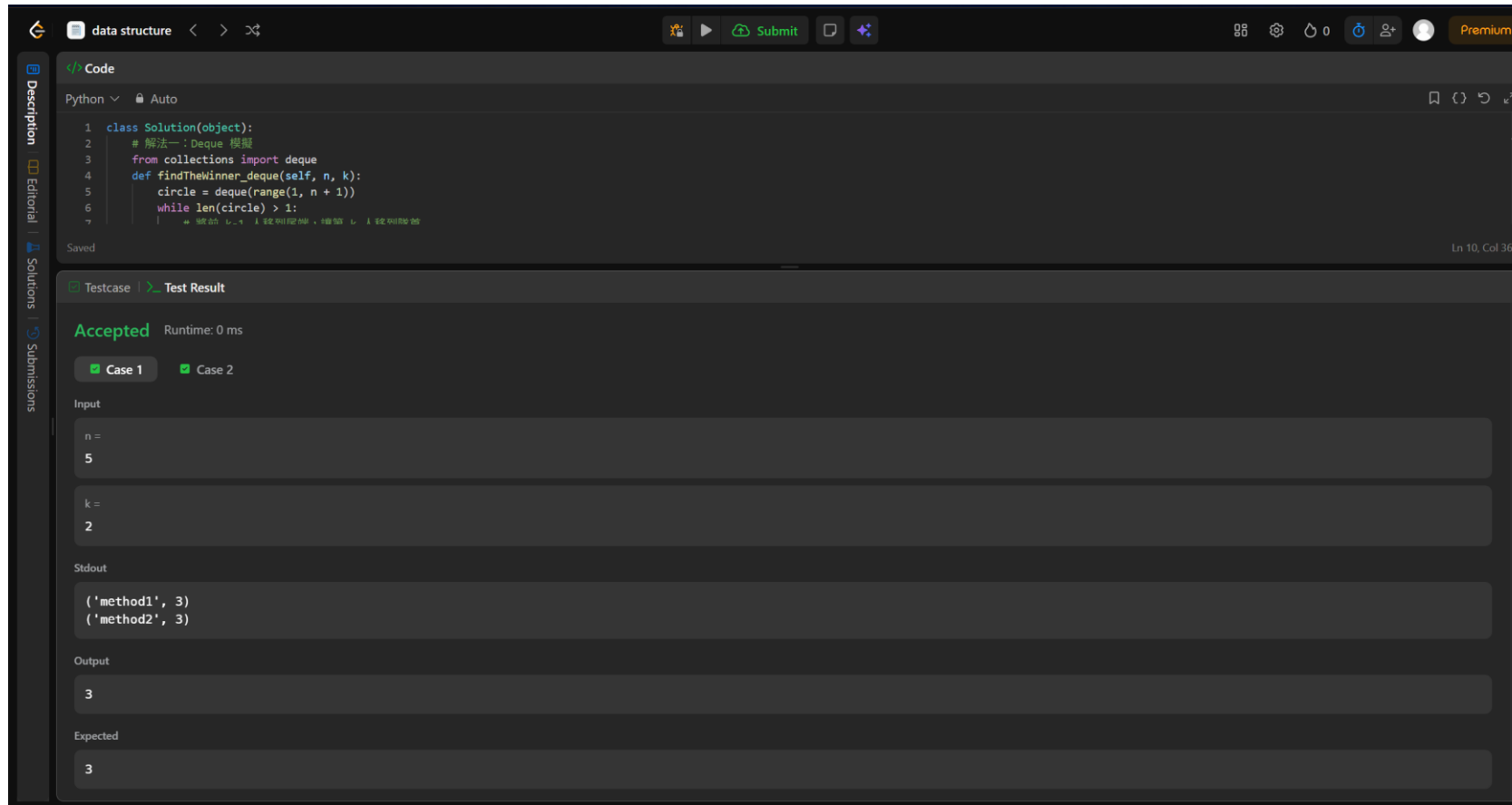
```

1 class Solution(object):
2     # 解法一：Deque 模擬
3     from collections import deque
4     def findTheWinner_deque(self, n, k):
5         circle = deque(range(1, n + 1))
6         while len(circle) > 1:
7             # 將前 k-1 人移到尾端，讓第 k 人移到隊首
8             # 這樣 popleft 就能精確移除第 k 個人
9             # 用 rotate 而非手動搬移是為了避免 off-by-one 錯誤
10            circle.rotate(-(k - 1))
11            circle.popleft()
12        return circle[0]
13
14    # 解法二：約瑟夫公式 O(n)
15    def findTheWinner_josephus(self, n, k):
16        # 從 1 人的情況開始推：只有 1 人時存活者索引必為 0
17        result = 0
18        for i in range(2, n + 1):
19            # 每多一人，圈子就大一圈
20            # 原本的索引在 i 人的圈中需要旋轉 k 格才是正確位置
21            # 對 i 取模是因為圈子大小為 i，超過就繞回去
22            result = (result + k) % i
23        # 全程用 0-indexed 計算是因為取模從 0 開始最自然
24        # 最後加 1 轉回題目要求的 1-indexed
25        return result + 1
26
27    def findTheWinner(self, n, k):
28        res1 = self.findTheWinner_deque(n, k)
29        res2 = self.findTheWinner_josephus(n, k)
30
31        print("method1", res1)
32        print("method2", res2)
33
34        return res1
35

```



- 結果證明：



The screenshot shows a code editor interface with a dark theme. The top bar includes a file explorer, a search bar, and a 'Submit' button. The main editor area displays Python code for a class named 'Solution'. The code implements a method 'findTheWinner_deque' that uses a deque to simulate a game. The code is as follows:

```
1 class Solution(object):
2     # 解法一: Deque 模擬
3     from collections import deque
4     def findTheWinner_deque(self, n, k):
5         circle = deque(range(1, n + 1))
6         while len(circle) > 1:
7             # 每次刪除 k-1 個元素，然後刪除第 k 個元素
```

Below the code editor, the 'Test Result' section shows that the solution was 'Accepted' with a runtime of 0 ms. It lists two test cases, 'Case 1' and 'Case 2', both of which passed. The input for Case 1 is n=5 and k=2. The stdout shows the output of the methods, and the output is 3, which matches the expected result.



LeetCode編碼：

649

系所：北聯(北醫) 生物醫學工程學系
學號：B812111001
姓名：張博翔



題目 (含解釋&說明)



- 字串 senate 由 ‘R’ (Radiant) 與 ‘D’ (Dire) 組成，為兩黨參議員的出場順序。輪到參議員時，可以選擇禁止對方陣營中尚未行動的任一人使其本輪失去行動權，或在對方已全部被禁止時宣布己方獲勝。按字串順序行動，本輪結束後循環進入下一輪，求哪一方最終勝出。
- 以 senate = “RDDRD” 為例。
 R[0] 最先行動，為了讓己方盡量保留人數最佳策略是禁止對方最早出現的人，所以禁止 D[1]。
 D[1] 被禁，D[2] 行動，禁止 R[0]。
 R[3] 行動，禁止 D[4]。
 進入第二輪，成員只剩 D[2] 與 R[3]，但 R[3] 已被 D[2] 的禁止，Dire 獲勝。
 最優策略是禁止對方最早出現的人，因為順序越早的人越快取得行動權，若不優先禁止他，他就會先禁止己方的人，己方在人數上就先吃虧。
 因此雙方的最優策略都是貪心禁止對方佇列中的頭部，使用雙佇列解法。
 跨輪比較時同一人下一輪的索引需加 n ，目的是讓索引大小能跨輪次統一比較先後順序，避免下一輪的人被誤判為比這輪晚出現的人還早行動。



- 解法一： **Double Queue**

- 既然最優策略都是禁止對方最早出現的人，那麼只要讓雙方各自維護一個佇列，存放尚未被禁止的人的出場索引，每輪比較雙方頭部的索引大小，索引較小的人代表先行動，可禁止對方的頭部。
- 行動者索引加上 n 後放回隊尾， $+n$ 是為了解決同一個人進入下一輪後，索引仍然大於本輪所有人，確保跨輪次的先後順序在比較時依然正確。

先後由索引大小決定，不需要額外 **ban** 計數或輪次管理。缺點是初始化時需要 **enumerate** 分別建立兩個佇列。



- 解法二：Queue + 計數
- 以 **banR** 與 **banD** 兩個計數器追蹤各方累積的禁令數，當行動時先檢查這方是否有累積的禁令。
若有代表對方在更早的時候就已經要禁止你，跳過且己方人數減 1；
若沒有則對方的 **ban** 計數加 1，代表行動了一次，禁止了對方的下一個人，然後回隊尾繼續參與後續輪次。
- 只需要記住對方欠我一個禁令，等對方的人出現時消耗，整個流程可以用單一佇列完成，省去雙佇列的索引管理。
優點是整體架構更簡潔，不需處理索引偏移。缺點是 **ban** 計數是類似抽象的欠帳概念，失去了誰對誰禁止的直觀性，且消耗禁令時必須同時將己方人數減 1 並 **continue** 跳過 **append**，否則被禁者仍會留在佇列中行動。
- 索引較小的人一定先禁止對方，因此最優策略就是禁止對方佇列中最早出現的人。
兩種解法本質相同，差別只在於順序是用索引大小還是 **ban** 計數來追蹤。
以佇列順序模擬優先順序的設計，在排程類題目中為常見手法，如 **LeetCode 2402**。



方法	Time	Space
雙佇列	$O(n)$	$O(n)$
單佇列+ban	$O(n)$	$O(n)$



```

1 class Solution(object):
2     from collections import deque
3
4     # 解法一：雙佇列模擬 (Two Queues)
5     def predictPartyVictory_two_queues(self, senate):
6         n = len(senate)
7         radiant = deque(i for i, c in enumerate(senate) if c == 'R')
8         dire = deque(i for i, c in enumerate(senate) if c == 'D')
9
10        while radiant and dire:
11            r, d = radiant.popleft(), dire.popleft()
12            # 索引較小者代表行動順序在前，可以禁止對方
13            if r < d:
14                # R 禁止 D，R 繼續循環到下一輪 (索引+n表示進入下一輪)
15                radiant.append(r + n)
16            else:
17                dire.append(d + n)
18        return "Radiant" if radiant else "Dire"
19
20

```

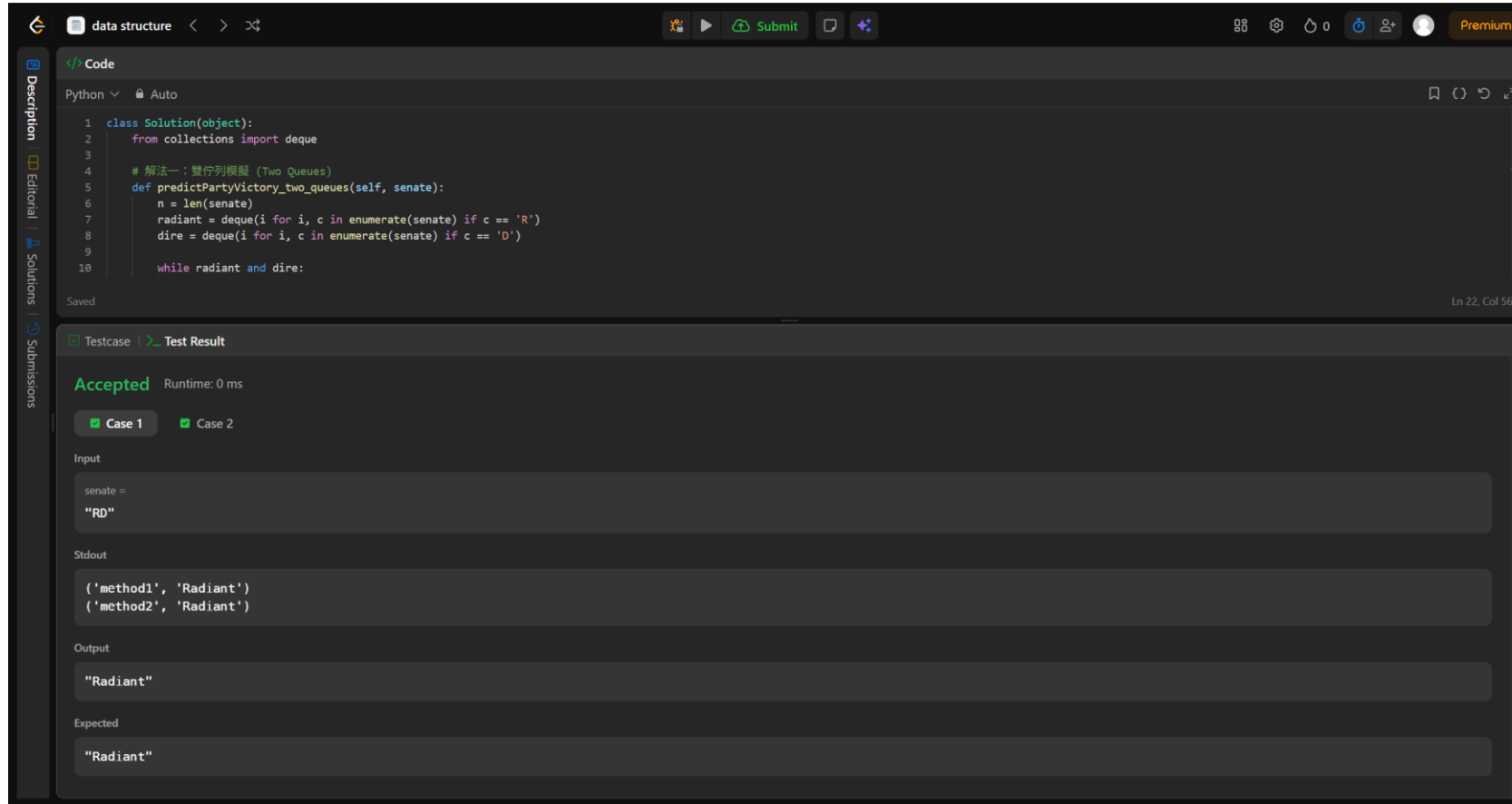
```

21 # 解法二：單佇列 + Ban計數 - 更精簡寫法
22 def predictPartyVictory_single_queue(self, senate):
23     n = len(senate)
24     queue = deque(senate)
25     banR, banD = 0, 0 # 累積對 R/D 的禁令數
26     countR = senate.count('R')
27     countD = n - countR
28
29     while countR > 0 and countD > 0:
30         cur = queue.popleft()
31         if cur == 'R':
32             if banR > 0:
33                 banR -= 1 # 此人被前面的D禁止，消耗一次禁令
34                 countR -= 1
35                 continue
36             banD += 1 # 此人禁止一名D議員
37         else: # cur == 'D'
38             if banD > 0:
39                 banD -= 1
40                 countD -= 1
41                 continue
42             banR += 1
43         queue.append(cur) # 未被禁止，放回隊尾繼續下一輪
44
45     return "Radiant" if countR > 0 else "Dire"
46
47 def predictPartyVictory(self, senate):
48     res1 = self.predictPartyVictory_two_queues(senate)
49     res2 = self.predictPartyVictory_single_queue(senate)
50
51     print("method1", res1)
52     print("method2", res2)
53
54     return res1

```



- 結果證明：



```

1 class Solution(object):
2     from collections import deque
3
4     # 解法一：雙佇列模擬 (Two Queues)
5     def predictPartyVictory_two_queues(self, senate):
6         n = len(senate)
7         radiant = deque(i for i, c in enumerate(senate) if c == 'R')
8         dire = deque(i for i, c in enumerate(senate) if c == 'D')
9
10        while radiant and dire:

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

senate =
"RD"

Stdout

('method1', 'Radiant')
('method2', 'Radiant')

Output

"Radiant"

Expected

"Radiant"